

Software Requirements Specification

SQL Code Generator

Afrem Jasmine-Emilia si Bimbirica Andra
IR3 2026, grupa 2

Contents

1	Introdúcere	2
2	Scop Si Tematica	2
3	Actori	2
4	Cerinte Functionale	2
5	Cerinte Nefunctionale	3
6	Structura Mesajelor	4
7	Fluxuri FCE	4
7.1	Upload Schema ER	4
7.2	Generare SQL	5
7.3	Validare Insert	5
7.4	Administramre	5
8	Configurare, CLI Si Mediu	5
9	Specificatie OpenAPI	6
10	Limitari Asumate	7
11	Livrabile Milestone 1	7

1 Introducere

SQL Code Generator este aplicatia aleasa pentru Milestone 1. Proiectul porneste de la un skeleton client-server C si dezvoltata o unealta care transforma o descriere ER in SQL si valideaza preventiv batch-uri `INSERT`.

Problema urmarita este una practica: inainte de rularea unor inserturi intr-o baza de date, serverul poate spune daca acestea ar incalca schema incarcata. Pentru demo, starea este tinuta in memorie partajata, nu intr-un SGBD real.

2 Scop Si Tematica

Aplicatia trebuie sa:

- primeasca o schema ER in format JSON;
- construiasca modelul intern de tabele, coloane si relatii;
- genereze instructiuni `CREATE TABLE`;
- valideze `INSERT INTO ... VALUES ...` pentru constrangeri uzuale;
- ofere rapoarte administrative printr-un client separat;
- demonstreze folosirea apelurilor UNIX cerute la curs: `socket-uri`, `poll`, `thread worker`, `fork`, `waitpid`, `mmap`;
- integreze biblioteci externe: `libconfig` pentru configurare si `ncurses` pentru interfata admin.

3 Actori

- Client ordinar: incarca schema ER, cere generarea SQL si trimite inserturi pentru validare.
- Administrator: consulta starea serverului prin clientul `ncurses`.
- Server: accepta conexiuni, tine starea comuna si proceseaza cereri printr-o coada FIFO.

4 Cerinte Functionale

- Serverul asculta conexiuni TCP pentru clienti ordinari pe portul configurabil 18081.
- Serverul asculta conexiuni TCP admin pe portul configurabil 18082.
- Serverul accepta un singur client admin conectat simultan.

- Conexiunea admin expira dupa timeout configurabil, implicit 60 secunde.
- Clientii ordinari primesc un `client_id` prin operatia `OP_CONNECT`.
- Upload-ul schemei ER se face chunked, cu bucati de maxim 64 KB.
- Serverul parseaza schema ER din JSON si actualizeaza starea interna.
- Serverul genereaza SQL `CREATE TABLE` cu `PRIMARY KEY`, `UNIQUE`, `NOT NULL` si `REFERENCES`.
- Serverul valideaza batch-uri `INSERT` pentru tabele existente, coloane existente, valori `NULL`, duplicate si foreign key lipsa.
- Pentru validarea inserturilor, serverul creeaza un proces copil cu `fork()` si asteapta rezultatul cu `waitpid()`.
- Dupa validare reusita, procesul parinte aplica inserturile in starea partajata.
- Adminul poate cere rapoarte despre clienti, comenzi, durata medie, istoric, tabele si adancimea cozii.

5 Cerinte Nefunctionale

- Implementarea este in C pe UNIX/Linux.
- Comunicarea foloseste socket-uri TCP si protocol binar propriu.
- Multiplexarea conexiunilor se face cu `poll`.
- Cererile clientilor ordinari sunt puse intr-o coada FIFO procesata de un thread worker.
- Starea demonstrativa este alocata cu `mmap(..., MAP_SHARED | MAP_ANONYMOUS, ...)`.
- Codul trebuie sa ramana compilabil cu `-Wall -Wextra -std=c11`, fara warning-uri de build.
- Codul este comentat in romana, cu antet de fisier si explicatii pentru structuri, functii si blocuri de control importante.
- Pentru Milestone 1, demo-ul trebuie rulat pe un sistem care are instalat `libconfig-dev` si `libncurses-dev`, astfel incat integrarea bibliotecilor externe sa fie efectiva.

6 Structura Mesajelor

Fiecare mesaj are header fix:

```
typedef struct {
    uint32_t msg_size;
    uint32_t client_id;
    uint32_t op_id;
    uint32_t flags;
} MsgHeader;
```

Campurile sunt trimise in network byte order. `msg_size` descrie payload-ul, iar payload-ul poate fi text UTF-8 sau bytes de fisier.

Operatii principale:

- `OP_CONNECT`: inregistreaza un client ordinar si intoarce `client_id`.
- `OP_BYE`: inchide conexiunea ordinara.
- `OP_UPLOAD_BEGIN`: initializeaza fisierul temporar de upload.
- `OP_UPLOAD_CHUNK`: trimite un chunk din schema ER.
- `OP_UPLOAD_END`: finalizeaza upload-ul si parseaza schema.
- `OP_GENERATE_SQL`: cere SQL-ul generat din schema curenta.
- `OP_VALIDATE_INSERT`: valideaza si aplica un batch de inserturi.
- `OP_ADMIN_LOGIN`: autentifica adminul cu token demonstrativ.
- `OP_ADMIN_REPORT`: cere un raport admin.
- `OP_ADMIN_BYE`: inchide conexiunea admin.

Raspunsurile folosesc `OP_OK` sau `OP_ERROR` si payload text.

7 Fluxuri FCE

7.1 Upload Schema ER

1. Clientul trimite `OP_CONNECT`.
2. Serverul raspunde cu `client_id`.
3. Clientul trimite `OP_UPLOAD_BEGIN`.
4. Clientul trimite unul sau mai multe mesaje `OP_UPLOAD_CHUNK`.
5. Clientul trimite `OP_UPLOAD_END`.
6. Serverul parseaza JSON-ul si incarca tabellele in `SharedState`.
7. Serverul raspunde cu `OP_OK` sau `OP_ERROR`.

7.2 Generare SQL

1. Clientul trimite `OP_GENERATE_SQL`.
2. Serverul citește modelul ER din starea partajată.
3. Serverul construiește textul `CREATE TABLE`.
4. Clientul primește SQL-ul generat.

7.3 Validare Insert

1. Clientul trimite `OP_VALIDATE_INSERT` cu batch-ul SQL.
2. Serverul pune cererea în coada FIFO.
3. Worker-ul scoate cererea din coada.
4. Serverul creează proces copil cu `fork()`.
5. Copilul validează batch-ul pe starea partajată.
6. Parintele așteaptă cu `waitpid()`.
7. Dacă validarea reușește, parintele aplică randurile în `SharedState`.
8. Serverul răspunde cu succes sau eroarea de constraint.

7.4 Administrare

1. Adminul se conectează la portul admin.
2. Trimite `OP_ADMIN_LOGIN`.
3. Trimite `OP_ADMIN_REPORT` cu una dintre categoriile `clients`, `commands`, `avg`, `history`, `tables`, `queue`.
4. Serverul întoarce raport text.
5. La inactivitate peste timeout, serverul închide conexiunea.

8 Configurare, CLI Si Mediu

Serverul citește configurația în această ordine:

1. valori implicite din cod;
2. fișier `libconfig`, implicit `config.cfg`;
3. variabile de mediu;

4. argumente CLI.

Fisier libconfig:

```
server: {  
    port = 18081;  
    admin_port = 18082;  
    admin_timeout = 60;  
};
```

Variabile de mediu:

- CONFIG_PATH: cale alternativa pentru fisierul de configurare;
- SERVER_PORT: port clienti ordinari;
- ADMIN_PORT: port admin;
- ADMIN_TIMEOUT: timeout admin in secunde.

Argumente server:

```
./server --config config.cfg --port 18081 --admin-port 18082 --admin-timeout 60
```

Argumente client:

```
./client --host 127.0.0.1 --port 18081 --input examples/er_schema.json --generate \  
    --insert-file examples/inserts_ok.sql --no-repl
```

Argumente admin:

```
./admin --host 127.0.0.1 --port 18082
```

9 Specificatie OpenAPI

Protocolul implementat in Milestone 1 este binar peste TCP. Pentru cerinta de specificatii S/R/proto si pentru o eventuala extensie Web Service, documentul docs/openapi.yaml descrie o interfata HTTP/WS echivalenta:

- upload schema ER;
- generare SQL;
- validare insert;
- rapoarte admin;
- stream WebSocket pentru operatii bazate pe aceleasi op_id.

Aceasta specificatie este documentatie de proiect pentru nivelul Web Service; executabilele curente folosesc protocolul TCP descris mai sus.

10 Limitari Asumate

- JSON-ul ER acceptat este formatul din `examples/er_schema.json`.
- Parserul JSON este minimal si orientat pe demo.
- Parserul SQL accepta `INSERT INTO table [(cols...)] VALUES (...), (...);`.
- Starea demonstrativa are limite fixe: 32 tabele, 32 coloane/tabel, 1024 randuri/tabel.
- Autentificarea admin foloseste token static `admin`, suficient pentru demo-ul Milestone 1.

11 Livrabile Milestone 1

- SRS/SDD: `docs/SRS.pdf` si `docs/SDD.pdf`.
- Specificatie OpenAPI: `docs/openapi.yaml`.
- Demo practic: `server`, `client`, `admin`, exemplele JSON si SQL din directorul `examples`.
- Integrare lib externa: `libconfig` pentru configurare si `ncurses` pentru clientul admin.
- Folosire apeluri UNIX/sistem: `socket-uri` TCP, `poll`, `mmap`, `fork`, `waitpid`, `pipe`, `read/write`, `close`, `mkdir`, plus thread worker cu `pthread`.
- Parsing CLI si mediu: serverul accepta optiuni de configurare, iar clientii accepta `host/port` si optiuni de demo.
- Calitate cod: build-ul se verifica prin `make -B all`, iar analiza statica poate fi rulata cu `clang-tidy` daca este disponibila.